

Observations Document

1. Approach

I built a Python command-line tool that combines Playwright with a vision-capable model. The tool starts from GitHub, takes a screenshot of the current browser state, and asks the navigation model to choose one next action. The model can choose actions such as click, type, press, scroll, wait, back, extract, done, or fail. Navigation decisions are based on screenshots and recent history rather than GitHub CSS selectors, XPath, or hidden DOM structure.

The main pipeline is:

1. Open GitHub with Playwright.
2. Take a screenshot of the current page.
3. Ask the vision model for the next action.
4. Execute browser actions with Playwright.
5. If the action is extract, read the current page's visible text and send it to a separate extraction prompt.
6. Save the extraction result into `extraction_history`.
7. Continue until the model returns done, fail, or the step limit is reached.
8. Generate a final concise `final_output.json` from the extraction history.

2. What Worked Well

- Bbox-based clicking was more reliable than asking the model for a single click point. The model returns a bounding box around a visible target, and the code clicks the center of that box.
- Separating navigation from extraction made the prompts simpler and easier to debug. The navigation component only chooses actions; the extractor only summarizes page content.
- Saving screenshots and `run_log.json` made it much easier to inspect where the agent succeeded or failed during each run.
- CLI options such as `--repo` and `--prompt` made the tool usable beyond the default `openclaw/openclaw` task.

3. What Did Not Work Well Initially

- Allowing a `goto` action made the agent too willing to construct GitHub URLs directly. I removed `goto` from the final action set to keep the task closer to visual navigation.
- Early extraction attempts copied too much release text into JSON, which sometimes caused invalid JSON. I changed extraction to summarize long content instead of copying raw text.

4. Trade-offs

The biggest trade-off is between robustness and simplicity. A more advanced agent could maintain a richer state object, use retry-specific prompts or implement a planning layer. I kept the design simpler so that the code remains easy to inspect and reason about.

Another trade-off is that the agent prioritizes clarity and debuggability over speed. Each step is logged, and extraction is a separate model call, which adds latency but makes it easier to debug.

I also limited the amount of history passed to the navigation model. This helps control prompt length, but it means older context may be summarized only through extraction records and logs rather than full raw page text.

5. Error Handling and Edge Cases

- Model JSON parse or schema validation errors are logged and retried without consuming a navigation step.
- Extraction failures are recorded as `extraction_failed` entries instead of crashing the whole run.
- A max step limit prevents the agent from looping forever if it cannot find a missing field.
- Final output can be success, partial, or failed depending on how much useful information was extracted.

6. Limitations

- The agent is still dependent on model quality. If the model misreads a page or returns invalid JSON repeatedly, the run may fail.
- The agent uses visible page text for extraction, so content hidden behind collapsed UI may not be captured unless the navigation agent opens it first.
- The current navigation policy can sometimes waste steps by repeating actions on the same page, such as extracting the same page more than once.
- Scrolling is also not very precise. Since the model only gives a general scroll direction and distance, moving through a short or partially visible page may take more steps than necessary.

7. Future Improvements

Future work could focus on reliability, recovery, and broader evaluation. A state field could help the agent distinguish whether it is on the home page, search results page, repository page, Releases page, or release detail page. This would make recovery easier when the model clicks the wrong item or lands on an unexpected page.

The navigation and extraction flow could also become more targeted. If an extraction record says that asset links are missing or release notes are truncated, the agent could explicitly look for visible “Assets,” “Read more,” or release-title links before deciding to stop. Similarly, the agent could track whether it has already extracted the current page and avoid repeating extraction unless the page content has changed.

Scrolling could be improved by using page-aware feedback. Instead of relying only on the model to estimate a scroll distance, the browser controller could record scroll position, page height, and whether the visible content changed after scrolling. This would help prevent wasted steps when the page is short or when a scroll action has little effect.

Model-output handling could be improved with JSON repair prompts and action-specific retry logic. Instead of failing or retrying the whole step, the system could ask the model to correct only the malformed response. For longer tasks, extraction history could also be compressed into a running summary to keep prompts short.